🧩 The Goto Statement

Understanding Unconditional Branching in C Programming

🔀 Jumping 🏷️ Labels ⚠️ Best Practices

🧩 What is goto?

🔀 The **goto statement** in C provides a way to **jump directly to another point** in your program — anywhere within the same function.

Key Concept:

This is an **"unconditional jump"** — meaning control **immediately transfers** to the labeled statement, skipping all code in between.

💡 **Analogy:** Imagine you're reading a book, and there's a note that says: "Go to page 45 and continue reading there." That's exactly what goto does.

📜 Syntax

General Forms:

```
goto label;   // jump to label

...

label:
    statement;   // this is the target
```

- **Label**
  Any valid variable name, followed by a colon

- **Placement**
  Label is placed immediately before statement where control is to be transferred

🧠 How it Works Conceptually

Execution Flow:

1. Program executes sequentially until encountering **goto label;**
2. Control **immediately jumps** to the statement following **label:**
3. Execution continues from that point, **skipping all code in between**

🧩 **Known as:** "Jumping statement" - breaks normal sequential execution of program

💻 Example

```
#include<stdio.h>
#include<conio.h>
main() {
    int i = 1, j;
    clrscr();

    while(i <= 3) {
        for(j = 1; j <= 3; j++) {
            printf("I=%d \t J=%d \n", i, j);
            if(j == 2) goto stop;
        }
        i = i + 1;
    }

stop:
    getch();
}
```

Output:

```
I=1 J=1
I=1 J=2
```

🔍 Step-by-step Execution

1. **Start**
   Initialize i = 1, enter while(i <= 3) loop

2. **First For Loop**
   Enter for(j = 1; j <= 3) loop

3. **First Iteration**
   j = 1 → print "I=1 J=1"

4. **Second Iteration**
   j = 2 → print "I=1 J=2"

5. **Condition Met**
   if(j == 2) → goto stop;

6. **Jump**
   Program jumps to label stop: → exits both loops

⚙️ Why Use Goto?

🧩 The goto statement is useful to provide **branching within a loop**. When loops are deeply nested and an error occurs, it can be difficult to exit from such loops.

Problem with break

Simple break statement cannot work properly when loops are deeply nested.

A break only exits the innermost loop, not all nested loops.

Solution with goto

In this case, goto statement is used to exit multiple loops at once.

Provides immediate exit from any level of nesting.

💡 **Key Advantage:** Goto can break out of deeply nested loops when a simple break statement cannot work properly.

🌍 Real-world Use Case

💼 Imagine a **menu-driven program** that processes user input. If an error occurs deep inside nested loops or conditions, you may want to jump to a cleanup or error-handling block.

```
#include<stdio.h>

int main() {
    FILE *file = fopen("data.txt", "r");
    if (!file) {
        printf("Error opening file!\n");
        return 1;
    }

    int num;
    while (1) {
        if (fscanf(file, "%d", &num) != 1) {
            goto error;  // jump to cleanup if something goes wrong
        }

        printf("Read number: %d\n", num);
        if (num == 999) goto done;  // end early if special code found
    }

error:
    printf("An error occurred while reading file.\n");

done:
    fclose(file);
    printf("File closed. Exiting.\n");
```

```
    return 0;
}
```

Output (if file has corrupted data):

```
Read number: 10
Read number: 20
An error occurred while reading file.
File closed. Exiting.
```

⚠️ When Not to Use Goto

⚠️ While it's powerful, **goto is dangerous if misused** — it can make your code hard to read, debug, and maintain.

Problems with Goto

✗ Hard to read and debug

Better Alternatives

✓ break statement

✗ Hard to maintain

✗ Prone to logic errors

✗ Creates "spaghetti code"

✓ continue statement

✓ return statement

✓ Functions

💡 **Modern Practice:** Use structured control statements instead of goto for cleaner, more maintainable code.

🎯 Summary

When to Use Goto

• Breaking out of deeply nested loops

When to Avoid Goto

• When structured alternatives exist

- Error handling / cleanup
- Jumping to cleanup sections before exiting

- For simple control flow
- In modern codebases

Use goto sparingly and only when it makes the code clearer than alternatives!